

מגמת הנדסת תוכנה 5 יח"ל

Driver Moodle

מערכת לניהול שיעורי נהיגה

Web project databases and programming asynchronous services חלופה

שם התלמיד: אלי רפפורט

שם בית הספר: מקיף ז אשדוד

שם המנחה: אולגה גנדלמן

תאריך הגשה: 05/05/2026

במסגרת הפרויקט פיתחתי מערכת שרת-לקוח לניהול שיעורי נהיגה בשם Driver Moodle. המערכת נועדה לעזור לתלמיד, למורה ולמנהל לעבוד במקום אחד מסודר במקום להשתמש בהרבה הודעות, פתקים או קבצי אקסל.

המערכת כוללת שרת WCF מרכזי, בסיס נתונים Access, ושלושה לקוחות שונים: לקוח WPF למחשב, לקוח Web ב-Razor Pages, ולקוח MAUI שמתאים למובייל. כל הלקוחות עובדים מול אותו שירות, ולכן המידע נשמר באותו מקום ולא מתפזר בין כמה מערכות.

בפרויקט יש פעולות מרכזיות שמתאימות לעולם של לימודי נהיגה: הרשמה והתחברות, קביעת שיעור, ביטול שיעור, תשלום על שיעורים, צפייה בהתראות, שליחת הודעות, ניהול קורסים ופתיחת פניות תמיכה. לכל סוג משתמש יש מסכים שמתאימים לו, כדי שתלמיד, מורה ומנהל לא יעבדו באותו אופן.

בצד הנתונים השתמשתי בטבלאות כמו Student, Teacher, Lessons, Payments, Notifications ו-Tickets. דרך הטבלאות האלה המערכת שומרת תלמידים, מורים, שיעורים, תשלומים, הודעות ופניות. הגישה לבסיס הנתונים נעשית דרך שכבת ViewDB, כדי שהלקוחות לא יפנו ישירות ל-Access.

בצד האבטחה והבדיקות שמתי דגש על שמירת סיסמאות בצורה בטוחה יותר בעזרת PBKDF2, בדיקות קלט בסיסיות כמו שדות חובה ואימייל תקין, ושימוש בשאילתות עם פרמטרים כדי לא להכניס ערכים ישירות לתוך SQL. המטרה של הפרויקט הייתה לא רק לבנות מסכים, אלא לבנות מערכת שעובדת בצורה מלאה: משתמש נכנס, עושה פעולה במסך, השרת מקבל את הבקשה, הנתונים נשמרים בבסיס הנתונים, והמשתמש מקבל תשובה ברורה בחזרה. כך אפשר לראות את הקשר בין הלקוח, השרת ובסיס הנתונים.

תוכן עניינים

6	פרק 1: מבוא - ייזום ואפיון
8	פרק 2: Use Case ותרחיש שימוש
10	פרק 3: ארכיטקטורה, ממשקי משתמש ומפות ניווט
13	פרק 4: בסיס הנתונים
15	פרק 5: מבנה הפרויקט והמימוש
19	פרק 6: מדריך למשתמש
20	פרק 7: בדיקות ותיקונים
21	פרק 8: פירוט תהליכי עבודה
24	פרק 9: פירוט טכני נוסף
27	פרק 10: בדיקות מורחבות וקבצים מרכזיים
30	פרק 11: סיכום ונספחים

בדיקת דרישות המחון

כדי שיהיה קל לבדוק את העבודה, ריכזתי כאן בקצרה איפה כל דרישה מופיעה בפרויקט.

בוצע	דרישה	איפה זה מופיע בפרויקט
✓	בסיס נתונים יחסי	Access עם טבלאות Student, Teacher, Lessons, Payments, Notifications, Tickets ועוד.
✓	קשרים בין טבלאות	קשרים בין תלמיד, מורה, שיעור, תשלום, התראות וקורסים.
✓	שירות רשת	WCF עם IService1 ו-Service1.
✓	ארכיטקטורת שכבות	Client, Service, Model, ViewDB ובסיס נתונים.
✓	פעולות CRUD	הוספה, שליפה, עדכון וביטול/מחיקה דרך מחלקות ViewDB.
✓	ממשק משתמש WPF	לקוח WPF עם מסכים לתלמיד, מורה ומנהל.
✓	לקוח Web	דפי Razor Pages לפי תפקידי משתמש.
✓	לקוח MAUI	לקוח מובייל לפעולות מרכזיות.
✓	Use Case	תרחישי שימוש להתחברות, קביעת שיעור, תשלום, ביטול, התראות ותמיכה.
✓	Validation	בדיקות קלט בסיסיות: שדות חובה, אימייל, סיסמה ותאריכים.
✓	IValueConverter	שימוש בממירי ערכים בצד הלקוח להצגת נתונים.
✓	XAML Binding ו-XAML	שימוש במסכי XAML ופקדים להצגת נתונים.
✓	Async / Tasks	שימוש בפעולות אסינכרוניות כדי לא לתקוע את הממשק.
✓	אבטחת סיסמאות	שמירת סיסמאות בצורה בטוחה יותר בעזרת PBKDF2.
✓	הגנה מ-SQL Injection	שימוש בשאילתות עם פרמטרים במקום חיבור טקסט ישיר.
✓	הרשאות משתמש	הפרדה בין תלמיד, מורה ומנהל.
✓	התראות	שמירת התראות ושליפה לפי משתמש מקבל.
✓	דוחות ותשלומים	שמירת תשלומים ושליפת נתונים לצורך דוחות.
✓	App.config והגדרות רשת	מפורט בפרק 5 עם כתובת השירות, binding ו-basicHttpBinding.
✓	IService1 + Service1	מפורט בפרק 5 עם דוגמאות לפעולות שירות.
✓	ViewDB ו-BaseDB	מפורט בפרק 5, כולל חיבור ל-Access ושימוש בפרמטרים.
✓	תרשים ארכיטקטורה	מופיע בפרק 3 ומציג לקוחות, שירות, ViewDB ובסיס נתונים.
✓	הסבר הפעלה	מופיע בפרק 11: פתיחת הפרויקט, הרצת השירות והפעלת הלקוחות.

פרק 1: מבוא - ייזום ואפיון

Driver Moodle היא מערכת שרת-לקוח לניהול שיעורי נהיגה. בחרתי לעשות את הפרויקט הזה כי בזמן שלמדתי נהיגה הרגשתי שקביעת שיעורים לא תמיד מסודרת. לפעמים הכול היה בהודעות, לפעמים שעה השתנתה, ולא תמיד היה ברור מה נשאר לשלם.

המטרה שלי הייתה לבנות מערכת אחת שבה תלמיד, מורה ומנהל יכולים לעבוד מול אותו מידע. התלמיד רואה שיעורים ותשלומים, המורה מנהל תלמידים ושיעורים, והמנהל יכול לראות נתונים כלליים ולעזור כשיש בעיה.

המערכת בנויה משלושה לקוחות: WPF למחשב, Web ב-Razor Pages, ו-MAUI למובייל. כולם עובדים מול אותו שירות WCF ואותו בסיס נתונים Access.

מה הבעיה שהפרויקט פותר

- קביעת שיעורים דרך הודעות יכולה להיות לא מסודרת.
- קשה לעקוב אחרי שיעורים ששולמו ושיעורים שעדיין פתוחים.
- המורה והתלמיד לא תמיד רואים את אותה תמונה.
- אין מקום אחד שמרכז שיעורים, התראות, קורסים ופניות תמיכה.

קהל יעד

המערכת מיועדת לתלמידי נהיגה, מורי נהיגה ומנהלי בית ספר לנהיגה.

למה בחרתי דווקא בנושא הזה

בחרתי בנושא הזה כי הוא קרוב אליי. כשאני למדתי נהיגה, הרבה דברים היו תלויים בזיכרון ובהודעות. אם רציתי לבדוק מתי יש לי שיעור, הייתי צריך לחפש בשיחות. אם רציתי לדעת כמה שילמתי, זה לא תמיד היה במקום אחד.

בגלל זה ניסיתי לבנות מערכת שלא מסבכת את המשתמש. הרעיון הוא שכל פעולה חשובה תהיה במקום ברור: שיעורים במסך שיעורים, תשלומים במסך תשלומים, הודעות במסך התראות ופניות במסך תמיכה.

פרק 2: Use Case ותרשישי שימוש

במערכת יש שלושה סוגי משתמשים. כל אחד מהם מקבל מסכים שמתאימים לתפקיד שלו, כדי שלא תהיה גישה למסכים שלא קשורים אליו.

משתמש	מה הוא עושה במערכת
תלמיד	קובע שיעורים, רואה שיעורים קיימים, משלם, מקבל התראות ופונה לתמיכה.
מורה	מנהל תלמידים, שיעורים, הודעות, קורסים ודוחות.
מנהל	בודק נתונים כלליים, מטפל בפניות ובודק פעילות במערכת.

תרשישי שימוש מרכזיים

שחקן ראשי	Use Case	תיאור קצר
תלמיד	הרשמה והתחברות	התלמיד נרשם או נכנס למערכת לפי שם משתמש וסיסמה.
תלמיד	קביעת שיעור	התלמיד בוחר תאריך ושעה והשיעור נשמר בטבלת Lessons.
תלמיד	תשלום על שיעור	התלמיד משלם על שיעור והמערכת מעדכנת את Payments ואת Lessons.
תלמיד	צפייה בהתראות	התלמיד רואה הודעות שנשלחו אליו מהמורה או מהמערכת.
מורה	ניהול שיעורים	המורה רואה שיעורים ותלמידים ומעדכן נתונים לפי הצורך.
מורה	שליחת הודעה	המורה שולח הודעה או התראה לתלמיד.
מנהל	טיפול בפניות	המנהל רואה פניות תמיכה, עונה עליהן וסוגר אותן.

- הרשמה והתחברות: המשתמש מזין פרטים, המערכת בודקת תקינות ומזהה את התפקיד שלו.
- קביעת שיעור: תלמיד בוחר תאריך ושעה, והשיעור נשמר עם המורה שלו.
- ביטול שיעור: שיעור מסומן כמבוטל ונשלחת התראה למורה.
- תשלום: תלמיד משלם על שיעור, והמערכת מעדכנת את סטטוס התשלום.
- פנייה לתמיכה: משתמש פותח פנייה והמנהל יכול לענות ולסגור אותה.

זרימת מידע כללית

הלקוח שולח בקשה לשירות WCF. השירות בודק את הנתונים, פונה לשכבת ViewDB, ושם מתבצעת העבודה מול Access דרך OleDb. התשובה חוזרת ללקוח ומוצגת במסך המתאים.

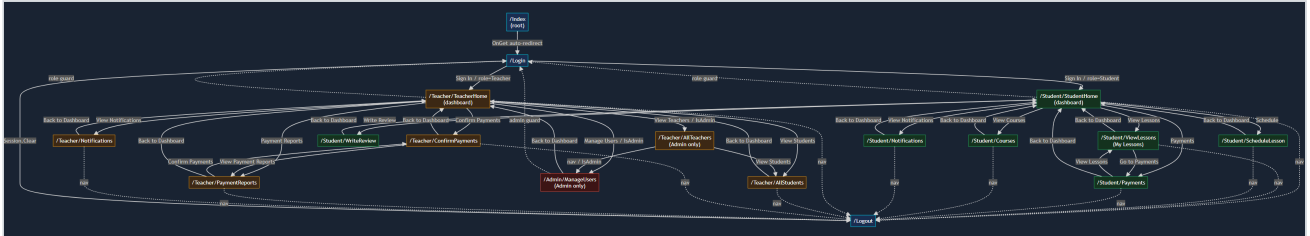
שלב	מה קורה בפועל
1	המשתמש לוחץ על פעולה במסך, למשל קביעת שיעור.
2	הלקוח בודק שהשדות הבסיסיים לא ריקים.
3	הלקוח שולח בקשה ל-Service1 דרך WCF.
4	השרת מבצע בדיקות נוספות ופונה למחלקת ViewDB המתאימה.
5	הנתונים נשמרים או נשלפים מ-Access.
6	התוצאה חוזרת למסך והמשתמש רואה הודעה או רשימה מעודכנת.

דרישות עיקריות מהמערכת

- המשתמש צריך להיכנס למערכת לפי שם משתמש וסיסמה.
- תלמיד צריך לראות רק את הנתונים שלו.
- מורה צריך לראות רק את התלמידים והשיעורים שקשורים אליו.
- מנהל צריך לקבל מסכים רחבים יותר לניהול ובקרה.
- כל פעולה חשובה צריכה להישמר בבסיס הנתונים ולא רק במסך.
- כאשר יש פעולה שמשפיעה על משתמש אחר, למשל ביטול שיעור, צריך ליצור התראה.

פרק 3: ארכיטקטורה, ממשקי משתמש ומפות ניווט

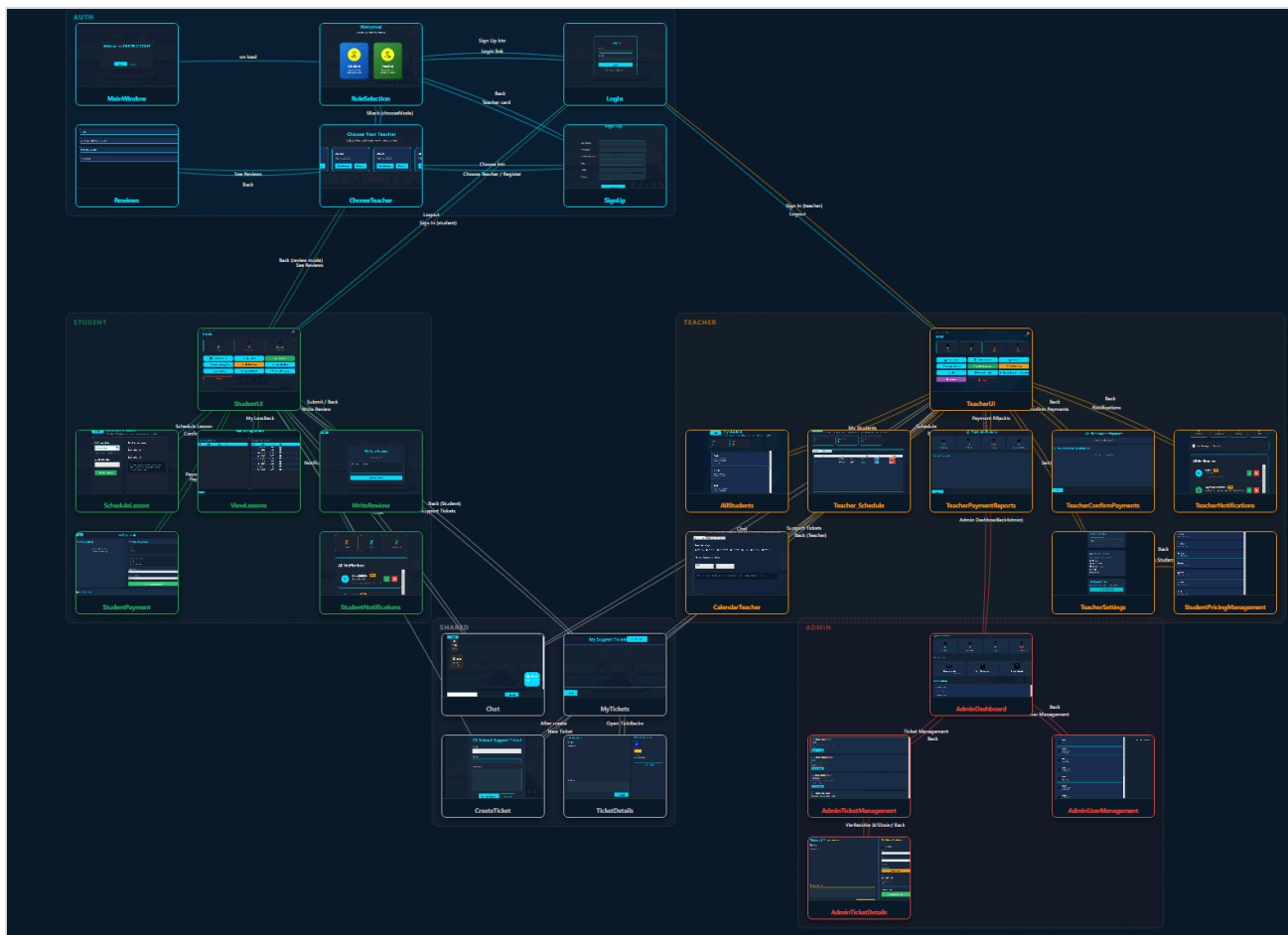
התרשים הבא מציג את המבנה הכללי של הפרויקט: שלושה לקוחות מתחברים לשירות WCF, השירות עובד מול שכבת ViewDB, ושכבת ViewDB עובדת מול בסיס הנתונים Access.



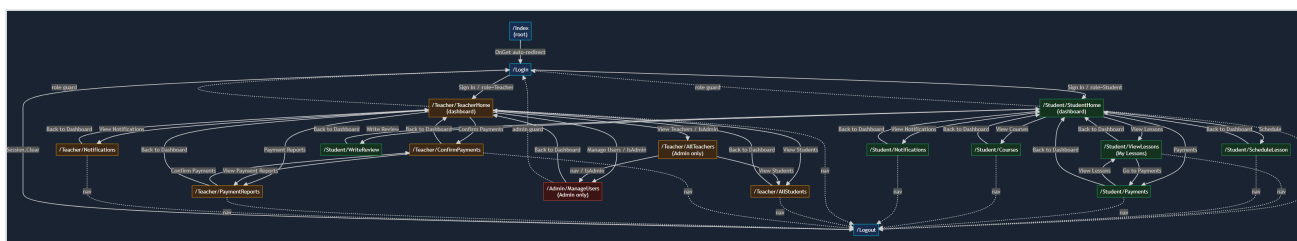
מבנה תיקיות מרכזי בפרויקט

תיקייה / קובץ	תפקיד
WcfServiceLibrary1/WcfServiceLibrary1	שירות WCF: IService1, Service1 ו-App.config.
WcfServiceLibrary1/Model	מחלקות הנתונים שעוברות בין הלקוח לשרת.
WcfServiceLibrary1/ViewDB	מחלקות גישה ל-Access כמו BaseDB, UserDB, LessonsDB ו-PaymentDB.
driver-client	לקוח WPF למחשב.
Driver/Driver/Pages	לקוח Web ב-Razor Pages.
driver-maui	לקוח MAUI למובייל.
nav-map	מפות ניווט ותרשימי מעבר בין מסכים.

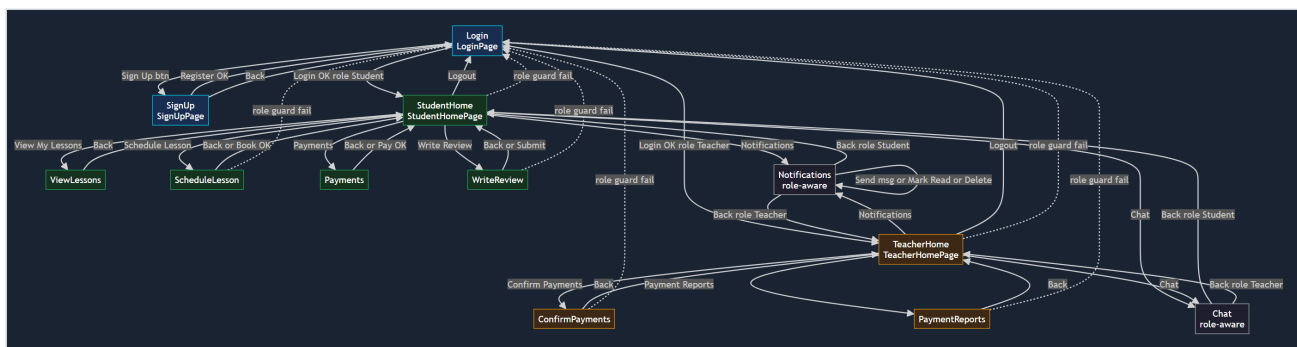
הכנתי מפות ניווט כדי שיהיה ברור איך עוברים בין המסכים בכל לקוח. לא כל לקוח נראה אותו דבר, אבל כולם עובדים מול אותו שרת.



מפת ניווט כללית של לקוח WPF



מפת ניווט כללית של לקוח Web



מפת ניווט כללית של לקוח MAUI

ב-WPF רוב העבודה מתבצעת דרך חלונות ומסכים. ב-Web העבודה מחולקת לדפים לפי תפקיד. ב-MAUI השתמשתי בניווט שמתאים יותר למובייל.

הסבר על הניווט ב-WPF

ב-WPF המשתמש מתחיל במסך התחברות או בחירת תפקיד. אחרי התחברות, המערכת מעבירה אותו למסך שמתאים לו. תלמיד מגיע למסכי תלמיד, מורה מגיע למסכי מורה, ומנהל מקבל אפשרויות ניהול.

היה לי חשוב שהמשתמש לא יצטרך לנחש לאן ללכת. לכן המסכים מחולקים לפי פעולות: שיעורים, תשלומים, התראות, קורסים ותמיכה.

הסבר על הניווט ב-Web

ב-Web השתמשתי בדפים נפרדים. זה מתאים יותר למסך ניהול, כי אפשר לפתוח דף לכל פעולה. גם כאן יש בדיקה לפי תפקיד, כדי שתלמיד לא ייכנס בטעות לדף שמיועד למורה או למנהל.

הסבר על הניווט ב-MAUI

ב-MAUI המטרה הייתה לתת גרסה נוחה יותר למובייל. לא כל מסך חייב להיות זהה ל-WPF, אבל הפעולות המרכזיות נשארות דומות: כניסה, צפייה בשיעורים, תשלום והתראות.

פרק 4: בסיס הנתונים

בסיס הנתונים נשמר בקובץ Access מסוג accdb. השתמשתי בכמה טבלאות כדי לא לשמור הכול במקום אחד. כך קל יותר לעדכן שיעור, תשלום, תלמיד או התראה בלי לפגוע בשאר הנתונים.

טבלה	תפקיד
Student	שמירת פרטי תלמידים וקישור למורה.
Teacher	שמירת פרטי מורים ונתוני עבודה.
Lessons	שמירת שיעורי נהיגה, תאריך, שעה, תלמיד ומורה.
Payments	שמירת תשלומים על שיעורים.
Notifications	שמירת הודעות והתראות למשתמשים.
Tickets	שמירת פניות תמיכה ותגובות.
Courses / Modules	שמירת קורסים, מודולים והתקדמות תלמיד.

קשרים בין טבלאות

קשר	הסבר
Student.teacherId -> Teacher.id	כל תלמיד משויך למורה נהיגה.
Lessons.StudentID -> Student.id	כל שיעור שייך לתלמיד.
Lessons.TeacherID -> Teacher.id	כל שיעור שייך גם למורה.
Payments.LessonId -> Lessons.LessonID	תשלום מקושר לשיעור מסוים.
Notifications.RecipientId	התראה נשמרת לפי המשתמש המקבל וסוג המשתמש.
StudentCourseProgress.ModuleId -> CourseModules.ModuleId	שמירת התקדמות תלמיד במודולים של קורס.

- כל תלמיד מקושר למורה.
- כל שיעור מקושר לתלמיד ולמורה.
- כל תשלום מקושר לשיעור ולתלמיד.
- התראה נשמרת לפי משתמש מקבל וסוג משתמש.

גישה לבסיס הנתונים

הגישה ל-Access נעשית דרך OleDb. המחלקות בשכבת ViewDB אחראיות על SELECT, INSERT, UPDATE ו-DELETE. הלקוח עצמו לא פונה ישירות לבסיס הנתונים.

```
Provider=Microsoft.ACE.OLEDB.12.0;Data Source=UsersDataBase.accdb
```

דוגמאות לשדות חשובים

טבלה	שדות לדוגמה	למה הם משמשים
Student	id, username, password, teacherId	זיהוי תלמיד, התחברות וקישור למורה.
Teacher	id, username, price, rating	שמירת פרטי מורה, מחיר ודירוג.
Lessons	LessonID, StudentID, TeacherID, Date, Hour, Paid	שמירת שיעור והאם הוא שולם.
Payments	PaymentId, StudentID, LessonId, Amount	שמירת תשלום לפי תלמיד ושיעור.
Notifications	Id, RecipientId, Message, IsRead	שמירת התראות שנשלחות למשתמש.

למה לא שמרתי הכול בטבלה אחת

אם כל הנתונים היו בטבלה אחת, היה קשה לעדכן ולבדוק אותם. למשל, שיעור צריך לדעת מי התלמיד ומי המורה, אבל פרטי התלמיד עצמם לא צריכים להישמר שוב בכל שיעור. לכן חילקתי את הנתונים לטבלאות שונות וקישרתי ביניהן.

פעולות בסיסיות על הנתונים

- הוספה: יצירת תלמיד, שיעור, תשלום או התראה.
- שליפה: הצגת שיעורים, תלמידים, תשלומים ודוחות.
- עדכון: שינוי סטטוס שיעור, סימון תשלום או סימון התראה כנקראה.
- מחיקה או ביטול: מחיקה במקומות שמתאימים, ובמקרים אחרים סימון כפעולה שבוטלה.

פרק 5: מבנה הפרויקט והמימוש

הגדרות WCF בקובץ App.config

לפי ההנחיות, חשוב להראות איפה נמצאות הגדרות הרשת. בפרויקט ההגדרות נמצאות בקובץ App.config של שירות ה-WCF. שם מוגדר basicHttpBinding, כתובת השירות ונקודת ה-MEX של השירות.

```
<?xml version="1.0" encoding="utf-8" ?>
<configuration>
  <appSettings>
    <add key="aspnet:UseTaskFriendlySynchronizationContext" value="true" />
  </appSettings>
  <system.web>
    <compilation debug="true" />
  </system.web>
  <!-- When deploying the service library project, the content of the config file must be added to the
  host's
  app.config file. System.Configuration does not support config files for libraries. -->
  <system.serviceModel>
    <bindings>
      <basicHttpBinding>
        <binding name="BasicHttpBinding_IService1" />
      </basicHttpBinding>
    </bindings>
```

ממשק השירות IService1

IService1 מגדיר את הפעולות שהלקוחות יכולים לקרוא להן. כך הלקוח לא פונה ישירות לבסיס הנתונים, אלא מבקש פעולה מהשרת.

דוגמאות לפעולות: CancelLesson, AddLessonForStudent, CheckUserPassword, Pay, CreateSupportTicket, GetUserNotifications.

שכבת BaseDB ו-ViewDB

BaseDB אחראית על החיבור ל-Access ועל פעולות בסיסיות של שליפה. המחלקות האחרות בשכבת ViewDB יורשות/משתמשות במבנה הזה כדי לעבוד בצורה מסודרת מול הטבלאות.

```
using Model;
using System;
using System.Collections.Generic;
using System.Data;
using System.Data.OleDb;
namespace ViewDB
{
  public abstract class BaseDB
  {
    private static string connectionString = null;
    protected OleDbConnection connection;
    protected OleDbCommand command;
    protected OleDbDataReader reader;
    protected abstract Base NewEntity();
    public BaseDB()
    {
      connection = GetConnection();
      command = new OleDbCommand();
    }
  }
}
```

הפרויקט מחולק לשכבות כדי שיהיה ברור איפה נמצא כל חלק. הלקוחות אחראים על המסכים. השרת אחראי על הפעולות. שכבת ViewDB אחראית על בסיס הנתונים.

שכבה	מה נמצא בה
לקוח	WPF, Web ו-MAUI. כאן נמצאים המסכים והפעולות שהמשתמש רואה.
שירות WCF	IService1 ו-Service1. כאן נמצאות הפעולות שהלקוחות קוראים להן.
Model	מחלקות הנתונים כמו Student, Teacher, Lesson ו-Payment.
ViewDB	מחלקות שמבצעות פעולות מול Access.

דוגמה לפעולה במערכת

כאשר תלמיד קובע שיעור, הלקוח שולח בקשה לשרת. השרת מקבל את הנתונים, בודק אותם, ושומר את השיעור בטבלת Lessons. אחרי זה הלקוח מקבל תשובה ומציג את השיעור במסך.

פעולה	איפה היא מתחילה	איפה היא נשמרת
קביעת שיעור	מסך תלמיד ב-WPF / Web / MAUI	Lessons
תשלום	מסך תשלומים	Lessons וגם Payments
שליחת הודעה	מסך מורה או תמיכה	Notifications / Tickets
ניהול קורס	מסך מורה	Courses ו-CourseModules
דוח	מסך מורה או מנהל	שליפה וחישוב מהטבלאות

החזרה של השירות

IService1 הוא המקום שבו מוגדרות הפעולות שהלקוחות יכולים לקרוא להן. Service1 הוא המימוש של הפעולות האלה. כך הלקוח לא צריך לדעת איך בדיוק נשמרים הנתונים בבסיס הנתונים.

```
public interface IService1
{
    bool CheckUserPassword(string username, string password);
    bool AddLessonForStudent(Lesson lesson);
    List<Notification> GetUserNotifications(int userId, string userType);
}
```

מחלקות Model

מחלקה	תפקיד
Base	מחלקת בסיס עם Id משותף.
Student	מייצגת תלמיד במערכת.
UserInfo	מידע כללי על משתמשים.
Calendar	שמירת זמינות ושעות פעילות.
Course / CourseModule	קורסים ומודולים לתלמידים.
Notification	התראות למשתמשים.
SupportTicket	פניות תמיכה והודעות בתוך פנייה.

מחלקות ה-Model מייצגות את הנתונים שעוברים בין הלקוח לשרת. לדוגמה Student מייצג תלמיד, Lesson מייצג שיעור, ו-Payment מייצג תשלום. השתמשתי במחלקות כאלה כדי לא להעביר נתונים בצורה לא מסודרת.

שאלות SQL

בפריקט יש שאלות פשוטות וגם שאלות מורכבות יותר. לדוגמה: שליפת שיעורים לפי תלמיד, עדכון תשלום, חישוב הכנסות, ושליפה של התראות לפי משתמש.

```
SELECT * FROM Lessons WHERE StudentID = @StudentID
```

אבטחה ובדיקות קלט

לא שמרתי סיסמאות כטקסט רגיל. השתמשתי ב-PBKDF2 כדי לשמור סיסמה בצורה בטוחה יותר. בנוסף, בקלט מהמשתמש יש בדיקות כמו שדות חובה, אימייל תקין ותאריכים הגיוניים.

בשאלות השתמשתי בפרמטרים במקום לחבר טקסט ישירות לשאלתה. זה עוזר למנוע הכנסת קוד SQL דרך שדות הקלט.

דוגמה לשאלתה עם פרמטר

```
command.CommandText = "SELECT * FROM Student WHERE username=@username";  
command.Parameters.AddWithValue("@username", username);
```

בצורה הזאת הערך שהמשתמש מזין לא נכנס ישירות לתוך הטקסט של השאלתה, אלא נשלח כפרמטר.

פרק 6: מדריך למשתמש

תלמיד

- נכנס למערכת עם שם משתמש וסיסמה.
 - רואה שיעורים שנקבעו לו.
 - יכול לקבוע שיעור חדש אם יש שעה פנויה.
 - יכול לשלם על שיעורים ולראות היסטוריה.
 - מקבל התראות מהמורה או מהמערכת.
- לדוגמה, תלמיד שנכנס למערכת יכול לבדוק מתי השיעור הבא שלו, אם הוא כבר שילם עליו, ואם יש הודעה חדשה מהמורה. זה חוסך חיפוש בהודעות ומרכז את המידע במסך אחד.

מורה

- רואה את התלמידים שלו.
 - מנהל שיעורים ותשלומים.
 - שולח הודעות והתראות לתלמידים.
 - יכול לנהל קורסים ומודולים.
- המורה משתמש במערכת כדי לעקוב אחרי תלמידים ושיעורים. במקום לזכור ידנית מי שילם ומי צריך שיעור נוסף, הנתונים נשמרים במערכת ואפשר לשלוף אותם לפי הצורך.

מנהל

- רואה נתונים כלליים על המערכת.
 - מטפל בפניות תמיכה.
 - בודק פעילות משתמשים ועוזר בבעיות.
- המנהל לא עובד כמו תלמיד או מורה. הוא צריך תמונה רחבה יותר של המערכת, בעיקר בשביל טיפול בבעיות ובקרה כללית.
- פירוט קצר של מסכים במערכת
- מסך התחברות: מאפשר למשתמש להכניס שם משתמש וסיסמה. לאחר בדיקה, המערכת מעבירה אותו למסך המתאים לתפקיד שלו.

- מסך שיעורים: מציג שיעורים קיימים ומאפשר קביעת שיעור חדש או ביטול שיעור לפי ההרשאות.
- מסך תשלומים: מציג שיעורים שצריך לשלם עליהם ושומר את פעולת התשלום בבסיס הנתונים.
- מסך התראות: מציג הודעות שנשלחו למשתמש, למשל הודעה מהמורה או עדכון על שינוי בשיעור.
- מסך תמיכה: מאפשר לפתוח פנייה ולנהל שיחה על בעיה עד שהיא נסגרת.

הודעות שגיאיה ומשוב למשתמש

כאשר המשתמש מזין פרטים לא נכונים, המערכת לא אמורה לקרוס. היא צריכה להציג הודעה פשוטה וברורה. לדוגמה: אם הסיסמה קצרה מדי או אם שדה חובה ריק, המשתמש מקבל הודעה ויכול לתקן.

פרק 7: בדיקות ותיקונים

בדקתי את המערכת לפי פעולות שהמשתמשים אמורים לבצע באמת. המטרה הייתה לראות שהזרימה עובדת ולא רק שהקוד מתקמפל.

בדיקה	מה נבדק	תוצאה
התחברות	שם משתמש וסיסמה נכונים ושגויים.	המערכת מכניסה משתמש נכון וחוסמת פרטים שגויים.
קביעת שיעור	בחירת תאריך, שעה ומורה.	השיעור נשמר ומופיע ברשימה.
ביטול שיעור	סימון שיעור כמבוטל.	השיעור מתעדכן ונשלחת התראה.
תשלום	תשלום על שיעור פתוח.	סטטוס התשלום מתעדכן.
פניית תמיכה	פתיחת פנייה ותגובה.	הפנייה נשמרת ומוצגת למנהל.

תיקונים שבוצעו

- שיפרתי ניווט בין מסכים כדי שמשתמש לא יגיע למסך שלא מתאים לתפקיד שלו.
- סידרתי תצוגות שהיו נראות לא טוב בגדלים שונים של חלון.
- הוספתי בדיקות קלט כדי להפחית שגיאות בזמן שימוש.
- בדקתי שהודעות והתראות נשמרות בבסיס הנתונים ונשלפות לפי המשתמש הנכון.

בדיקות קצה שעשיתי

- ניסיון התחברות עם סיסמה לא נכונה.
 - ניסיון לשמור שיעור בלי לבחור תאריך.
 - ניסיון לשלם על שיעור שכבר סומן כשולם.
 - כניסה למסך שלא מתאים לתפקיד המשתמש.
 - שליחת פנייה ריקה לתמיכה.
- הבדיקות האלה חשובות כי הן בודקות שימוש אמיתי ולא רק מצב שבו המשתמש עושה הכול נכון.

מה עדיין אפשר לשפר

יש דברים שהייתי משפר אם היה יותר זמן: עיצוב מסכים אחיד יותר בכל הלקוחות, יותר דוחות למנהל, וחיבור ליומן חיצוני כדי שהתלמיד יוכל לקבל תזכורת גם מחוץ למערכת.

פרק 8: פירוט תהליכי עבודה

בחלק הזה אני מפרט יותר את התהליכים המרכזיים במערכת. המטרה היא להראות שהפרויקט לא בנוי רק ממסכים, אלא מתהליך שלם שמתחיל אצל המשתמש ומסתיים בשמירה או שליפה של נתונים.

תהליך התחברות והרשאות

המשתמש מכניס שם משתמש וסיסמה. הלקוח שולח את הפרטים לשרת WCF. השרת בודק את המשתמש מול בסיס הנתונים, ואם הפרטים נכונים הוא מחזיר תשובה ללקוח. לפי התפקיד של המשתמש, הלקוח מעביר אותו למסך מתאים.

תהליך קביעת שיעור

בתהליך קביעת שיעור התלמיד בוחר תאריך ושעה. המערכת בודקת שהנתונים הגיוניים, שומרת את השיעור בטבלת Lessons, ומציגה אותו ברשימת השיעורים של התלמיד והמורה.

תהליך תשלום

כאשר תלמיד משלם על שיעור, המערכת שומרת רשומת תשלום בטבלת Payments ומעדכנת את השיעור כך שיהיה ברור שהוא שולם. בצורה הזאת אפשר להציג היסטוריית תשלומים ולחשב דוחות.

תהליך ביטול שיעור

ביטול שיעור לא חייב למחוק את השיעור לגמרי. במקרים כאלה עדיף לסמן את השיעור כמבוטל, כדי שתישאר היסטוריה. בנוסף, המערכת יכולה ליצור התראה למורה כדי שהוא ידע שהשיעור השתנה.

תהליך התראות

התראה נשמרת בטבלת Notifications עם המשתמש המקבל, סוג המשתמש, תוכן ההודעה והאם היא נקראה. כך כל משתמש רואה רק את ההתראות שלו.

תהליך פניות תמיכה

פניית תמיכה מאפשרת למשתמש להסביר בעיה. הפנייה נשמרת בבסיס הנתונים, והמנהל יכול לראות אותה, לענות עליה ולסגור אותה. זה נותן למערכת מקום מסודר לטיפול בבעיות במקום הודעות חיצוניות.

פרק 9: פירוט טכני נוסף

בחלק הזה יש פירוט נוסף על הצד הטכני של הפרויקט, אבל בשפה פשוטה כדי שיהיה קל להבין מה כל שכבה עושה.

שכבת הלקוח

שכבת הלקוח כוללת את WPF, Web ו-MAUI. התפקיד שלה הוא להציג מסכים למשתמש ולקבל ממנו פעולות. הלקוח לא שומר נתונים בעצמו, אלא שולח בקשות לשרת.

שכבת השירות WCF

שכבת השירות היא הגשר בין הלקוחות לבין בסיס הנתונים. הלקוחות קוראים לפעולות שנמצאות ב-Service1, והמחלקה Service1 מבצעת את העבודה בפועל.

שכבת ViewDB

שכבת ViewDB אחראית על העבודה מול Access. כל מחלקה מטפלת בנושא אחר: תלמידים, מורים, שיעורים, תשלומים, התראות או פניות.

בדיקות קלט ואבטחה

בפרויקט יש בדיקות קלט כדי לא לאפשר שמירה של נתונים לא תקינים. בנוסף, הסיסמאות נשמרות בצורה בטוחה יותר בעזרת PBKDF2, והשימוש בפרמטרים בשאילתות עוזר להגן מפני הכנסת SQL דרך שדות קלט.

פרק 10: בדיקות מורחבות וקבצים מרכזיים

בחלק הזה אני מפרט עוד בדיקות וקבצים מרכזיים בפרויקט. זה עוזר להראות שהמערכת נבדקה לפי פעולות אמיתיות, ולא רק לפי כתיבת קוד.

בדיקות התחברות

בדיקה	מה נבדק	תוצאה צפויה
פרטים נכונים	כניסה של תלמיד או מורה.	המשתמש נכנס למסך המתאים.
סיסמה שגויה	הכנסת סיסמה לא נכונה.	המערכת מציגה שגיאה ולא מכניסה את המשתמש.
שדות ריקים	ניסיון התחברות ללא פרטים.	המערכת מבקשת להשלים נתונים.

בדיקות שיעורים ותשלומים

בדיקה	מה נבדק	למה זה חשוב
קביעת שיעור	שמירת שיעור חדש בטבלת Lessons.	כדי לוודא שהשיעור לא נשאר רק במסך.
ביטול שיעור	סימון שיעור כמבוטל.	כדי לשמור היסטוריה ולא למחוק מידע חשוב.
תשלום	שמירת תשלום ועדכון שיעור.	כדי לדעת אילו שיעורים שולמו.
דוח תשלומים	שליפת נתונים מטבלאות תשלומים ושיעורים.	כדי לבדוק שהחישובים מסתמכים על נתונים אמיתיים.

קבצים מרכזיים בפרויקט

קובץ או תיקייה	תפקיד
IService1.cs	מגדיר את הפעולות שהלקוחות יכולים לקרוא להן.
Service1.cs	מממש את הפעולות של השרת.
ViewDB	כולל מחלקות שעובדות מול Access.
Model	כולל מחלקות נתונים כמו Student, Teacher, Lesson ו-Payment.
driver-client	לקוח WPF למחשב.
Driver/Pages	דפי Web ב-Razor Pages.
driver-maui	לקוח MAUI למובייל.
nav-map	מפות ניווט ותרשימים של המסכים.

פרק 11: סיכום ונספחים

הסבר הפעלה קצר

כדי להריץ את הפרויקט פותחים את הפתרון ב-Visual Studio, מוודאים שקובץ UsersDataBase.accdb נמצא בתיקיית ViewDB, מריצים את שירות ה-WCF, ולאחר מכן מריצים את אחד הלקוחות: Web, WPF או MAUI. אם אין משתמש קיים, אפשר להירשם דרך מסך ההרשמה. לאחר התחברות המערכת מזהה אם המשתמש הוא תלמיד, מורה או מנהל ומעבירה אותו למסך המתאים.

מה מצורף כנספחים

הקוד המלא מצורף בתיקיות הפרויקט. בגוף הספר הוכנסו הסברים וקטעי קוד קצרים בלבד, כדי שהספר יישאר קריא ושהבחון יוכל לראות במהירות איפה כל דרישה ממומשת.

בפרויקט הזה למדתי איך לחבר כמה לקוחות שונים לאותו שרת, ואיך לשמור נתונים בצורה מסודרת בבסיס נתונים. החלק שהיה לי הכי מאתגר היה לגרום לכל הלקוחות לעבוד עם אותו שירות בלי לשבור תאימות.

אני חושב שהפרויקט מתאים לנושא שבחרתי כי הוא פותר בעיה אמיתית שנתקלתי בה בעצמי. אם הייתי ממשיך את הפרויקט, הייתי משפר עוד את עיצוב המסכים ומוסיף אפשרות לסנכרון עם יומן חיצוני.

מה מצורף בסוף

- קוד הפרויקט המלא נמצא בתיקיות הפרויקט ולא הועתק כולו לתוך הספר.

- הקבצים המרכזיים: Service1.cs, IService1.cs, מחלקות Model, מחלקות ViewDB, מסכי WPF, דפי Web ודפי MAUI.

- מפות הניווט נמצאות גם בתיקיית nav-map.

ביבליוגרפיה קצרה

- Microsoft Learn - WCF, WPF, Razor Pages, MAUI.

- חומרי הלימוד של מגמת הנדסת תוכנה.

- תיעוד Microsoft Access ו-OleDb.